Edge Collaborative Caching Based on Incentive-Driven D3QN Combined With User Preferences in UAV-Assisted Vehicular Networks

Geng Chen, *Member, IEEE*, Yuchen Han, Fei Shen, *Member, IEEE*, Qingtian Zeng, *Member, IEEE*, and Yu-Dong Zhang, *Senior Member, IEEE*

***Abstract*—Mobile Edge Cache allows CDN edge nodes to be deployed closer to users, reducing content transmission latency in Vehicular Networks (VANETs). However, how to effectively utilize the limited storage space of cache nodes is the main issue in current research. To address this problem, we propose an incentive-driven hierarchical collaborative caching algorithm based on D3QN combined with user preferences (PC-ID3QN). Firstly, we constructed a UAV-assisted vehicular content-centric network framework, designing different collaborative caching strategy for various layers based on user preferences. Secondly, we proposed a hierarchical incentive mechanism based on social priority to motivate users to participate in collaborative sharing, thereby enhancing the utilization of system caching resources. Next, we modeled the cache placement problem as a system utility maximization problem and proved its NP-hardness. Then, by adjusting the constraint conditions, we conducted theoretical analysis and transform it into a linear programming(LP) problem to obtain an offline theoretical solution. This solution was validated against the simulation optimal solution obtained using the proposed PC-ID3QN algorithm, demonstrating the effectiveness of the algorithm. Finally, we validate the proposed caching strategy using the MovieLens dataset and conduct extensive experiments to verify the applicability and superiority of our solution in improving cache utility. Compared with DDQN, Dueling DQN and DQN, our proposed ID3QN algorithm reduces the request delay by 1.03%,1.73% and 2.21%, and reduces the energy cost by 38.8%, 19.6% and 17.2%, respectively.**

Received 21 September 2024; revised 24 March 2025 and 30 June 2025; accepted 27 August 2025. Date of publication 8 September 2025; date of current version 1 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 61701284, Grant 52574256, and Grant 52374221; in part by the Natural Science Foundation of Shandong Province of China under Grant ZR2022MF226, Grant ZR2022MF288, and Grant ZR2023MF097; in part by the Talented Young Teachers Training Program of Shandong University of Science and Technology under Grant BJ20221101; in part by the Innovative Research Foundation of Qingdao under Grant 19-6-2-1-cg; in part by the Elite Plan Project of Shandong University of Science and Technology under Grant skr21-3-B-048; and in part by the Taishan Scholar Program of Shandong Province under Grant tstp20250506. The Associate Editor for this article was M. Shojafar. *(Corresponding authors: Geng Chen; Qingtian Zeng.)*

Geng Chen, Yuchen Han, and Qingtian Zeng are with the College of Electronic and Information Engineering, Shandong University of Science and Technology, Qingdao 266590, China (e-mail: gengchen@sdust.edu.cn; heanyc@163.com; qtzeng@sdust.edu.cn).

Fei Shen is with Shanghai Institute of Microsystem and Information Technology, Chinese Academy of Sciences, Shanghai 200050, China (e-mail: fei.shen@mail.sim.ac.cn).

Yu-Dong Zhang is with the School of Computing and Mathematical Sciences, University of Leicester, LE1 7RH Leicester, U.K. (e-mail: yudongzhang@ieee.org).

Digital Object Identifier 10.1109/TITS.2025.3604600

***Index Terms*—Edge caching, hierarchical collaboration, content matching, incentive-driven, D3QN.**

I. Introduction

With the rapid development of Intelligent Transportation Systems (ITS), data traffic and information exchange in transportation networks have surged. The development of Mobile Edge Computing (MEC) provides mobile network operators (MNOs) with the opportunity to deploy Content Delivery Networks (CDNs) deeper into mobile network infrastructure.

MEC enables CDN edge nodes to be placed closer to vehicular users, allowing faster access to requested services [1]. However, the limited resources of VEC systems pose challenges in effectively managing and optimizing, making the design of efficient caching strategies a key research issue [2].

To increase caching resources, by adding more cache nodes in vehicles, drones, and roadside infrastructure [3], [4], distributed caching resources can be leveraged to enhance content retrieval efficiency. Vehicle Edge Cache (VEC) has two basic communication modes: Vehicle-to-Vehicle (V2V) and Vehicle-to Infrastructure (V2I). However, compared to the ever-increasing data traffic, the available resources at server cache nodes are very limited. Therefore, efficient caching algorithms are needed.

In current research, caching can be divided into reactive caching and proactive caching based on the order of requests and caching [22], [23]. Proactive caching primarily involves using predictive models to anticipate content needs in advance [4]. Additionally, considering the homogeneity and heterogeneity of preferences among mobile vehicles [2], individual users have varying file preferences. Therefore, to maximize cooperation and content sharing among mobile vehicles, it is necessary to consider the homogeneity of user preferences when caching across different levels of edge cache nodes.

Driven by the concept of the Sharing Economy (SE), the surplus computing and caching resources in an edge computing framework can be shared by edge users, thereby gaining some additional benefits [6]. Therefore, an incentive caching scheme centered on CDNs in the IoV is particularly important. We combine the concept of social contribution rate from socioeconomics with caching efficiency, defined as the ratio of effectively cached content to the resources consumed.

On this basis, it is necessary to design a reasonable incentive mechanism for edge caching content to motivate vehicle users to provide content services to each other [7].

Based on the above, we propose a hierarchical collaborative caching algorithm based on D3QN that incorporates user preference-driven incentives (PC-ID3QN). This scheme integrates a user preference evaluation model and a caching model, allowing it to adaptively assess the importance of content in dynamic networks and address content matching across different hierarchical levels. Additionally, by incorporating a social contribution-based incentive model into the Double Deep Q-Network (D3QN), our approach fosters greater user engagement and cooperation by rewarding contributions to the system, which further optimizes cache space utilization and reduces delays. The main contributions of this paper are as follows.

1) We propose a hierarchical collaborative content caching and replacement strategy based on user preferences. This approach learns vehicle request preferences by combining the similarity between historically requested content. It selectively caches content to adapt to the dynamic and real-time updates of vehicle user request information, thereby improving content cache hit rates and enhancing cache space utilization efficiency.
2) We designed an incentive mechanism to encourage user participation in system collaboration, which effectively reduces content retrieval delays while enhancing cache space utilization. Thereby improving collaboration and content sharing efficiency among mobile vehicles.
3) We proposed a cache utility maximization problem to address content retrieval delays and energy consumption across different links, and proved that the problem is NP-hard, conducted a theoretical analysis and solution using the DCA algorithms. Additionally, we obtained simulation results under dynamic environment conditions, which were mutually validated. The model's effectiveness was validated through real trajectory-driven simulations.

The structure of the remainder of this paper is as follows: In Section II, we present related work. Section III introduces the system model. Section IV we formulate optimization problem. In Section V, we analyze the optimization problem and provides a detailed explanation of the D3QN reinforcement learning algorithm. Section VI evaluates the performance of the proposed caching algorithm through simulations. Section VII concludes the paper.

II. Related Work

In this section, we review the related work on edge caching. In formulating caching strategies, Hou et al. [8] focus solely on user request models set to Zipf distribution without considering user preferences or historical request information. Guan et al. [9] primarily addresses video content caching and proposes a predictive edge caching strategy called PrefCache, which predicts user preferences for video topics. However, the above studies focus solely on user preferences, overlooking caching resource allocation.

Hou et al. [8] proposes an efficient cache space control algorithm (CSCA) to help edge cache nodes determine the specific amount of cache space provided, minimizing caching costs to an acceptable level. Farahani et al. [32] proposed a latency- and cost-aware hybrid P2P-CDN framework (ALIVE) by integrating P2P and CDN nodes, which enhances Quality of Experience (QoE), reduces end-to-end latency, and improves network utilization. ARARAT [33] combines Software-Defined Networking (SDN), Network Function Virtualization (NFV), and edge computingto achieve efficient HTTP adaptive video streaming. These advanced caching strategies not only improve content distribution but also meet the growing demand for low-latency and high-reliability multimedia services in IoV environments. However, the above work focuses on cache resource allocation algorithms, neglecting the use of multiple cache nodes to expand capacity.

Recent research has introduced UAVs to assist in caching content between nodes. Han et al. [2] considers urban traffic congestion and designs a Space-Air-Ground Integrated Network (SAGIN), introducing unmanned aerial vehicles (UAVs) as data collectors to assist the intelligent transportation system. Liu et al. [3] proposes a Deep Q-Network (DQN)-based hybrid caching and replacement scheme in a scenario where UAV-assisted RSUs handle a large number of computational tasks for vehicle users. Chen et al. [20] proposes an IoT network supported by multiple satellites and multiple cache-assisted UAVs aims to optimize the total network latency. However, these studies only consider vertical collaboration between caching nodes, neglecting horizontal collaboration between vehicles and UAVs.

For research on collaboration between nodes at the horizontal level, Chaowei et al. [11] proposes a DRL-based cooperative caching scheme for vehicular edge networks with large-scale MIMO, optimizing caching decisions based on vehicle trajectories, speed, and user preferences. Li et al. [12] proposes a federated deep reinforcement learning (F-DRL)-based edge collaborative caching policy to alleviate the pressure on edge networks and protect privacy. However, these studies assume cooperative nodes, without considering whether nodes are selfish or willing.

To incentivize vehicle users to share cached content, Huang et al. [13] introduces a virtual currency system (VCS) to motivate user devices to share cached content with others. Yates [14] examines incentive issues for mobile users, focusing on which users are willing to cache, using evolutionary game theory to analyze strategies. Our work goes further by not only compensating for transmission costs but also determining service priority based on social contribution.

Unlike previous studies, we comprehensively considered the entire process of vehicular content pre-caching and replacement. This paper considers content caching strategies from the perspective of Content Provider (CP), caching content based on different popularity levels within a region to improve cache hit rates at edge nodes. Additionally, we introduces an incentive-driven approach called ID3QN, which incorporates the concept of social contribution to determine user service priority, to encourage nodes to participate in caching, thereby maximizing Quality of Service (QoS) in the network. Our

TABLE I
THE LIST OF MAIN NOTATIONS

Notation	Definition	Notation	Definition
V, U	collection of vehicles, drones in the scene	$P_{v,v'}, P_{u,u'}$	Transmission power between vehicles and between drones
V_n	collection of vehicles in the nth UAV coverage area	N_0	Noise power
F, W	contents set and Topic Set	$g_{vv'}(t)$	Channel gain between vehicles
v_m	Vehicles with serial number m	u_n	UAV with serial number n
s_f	size of content f	$l^2_{vv'}(t)$	Distance between vehicles
H^t_u, H^t_v	coordinates of drones, vehicles	σ	Gaussian white noise
φ_{v_m,f_k}	The preference level of vehicle user v for content f.	$R_{u,v}$	Transmission rate between V2U
c_{Mf_n}	Binary caching decision variable.	$R_{vv'}$	Transmission rate between V2V
C_u, C_v	The current size of the content cached by drones and vehicles.	$R_{u,u'}$	Transmission rate between U2U
C_M	Caching capacity constraint.	$E^f_{v,v'}, E^f_{b,v}$	The energy consumption through V2V V2B
$d_{u,u'}, d_{v,b}$	Transmission time between U2U V2B	E_{tra}	Transmission power of drones
p_f	The probability of a user requesting a file.	ϖ_v, ϖ_b	Energy consumption cost for transmission per unit time
ω_1, ω_2	Caching weight coefficients for different layers.	δ_f	Maximum tolerable delay for the task
g_0	The power gain per unit reference distance.	$B_{v,v'}, B_{u,u'}$	Transmission bandwidth between V2V and U2U

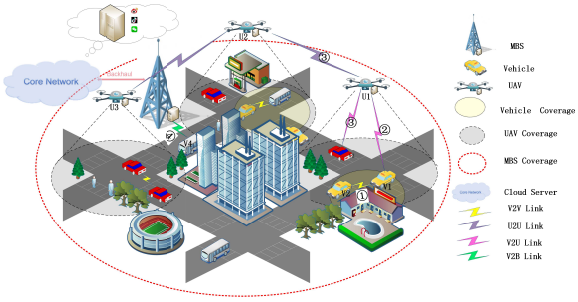


Fig. 1. UAV-assisted Telematics edge caching system.

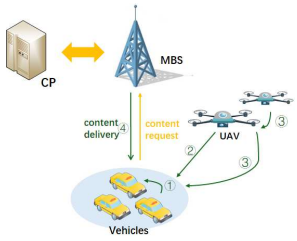


Fig. 2. Content request and delivery paths.

study not only considers cooperative caching between vehicles on a two-dimensional plane but also extends caching to a three-dimensional plane by introducing UAVs as aerial base stations.

III. SYSTEM MODEL

In this section, we construct a urban scenario and analyze it through the construction of a network model, a caching model, and a communication model. Table I provides a detailed explanation of the symbols used.

A. Network Model

Fig. 1 illustrates three types of edge caching nodes in vehicular edge computing networks: macro base stations (MBSs), UAVs $U = \{u_1, u_2, \ldots, u_n\}$, and mobile vehicles $V = \{v_1, v_2, \ldots, v_m\}$. Both UAVs and vehicles are equipped with limited caching storage capacity. The time period T is divided into multiple time slots, during which vehicles select content from the content library $\mathrm{F} = \{f_1, f_2, \ldots, f_k\}$ and issue content requests. The MBS, connected to the cloud server, receives content requests and serves all vehicles within its coverage area. As a central controller, the MBS manages all edge servers and synchronizes storage information. The size of each content item is denoted by s_k bits. The content in the cloud is provided and continuously updated by content providers.

During peak hours, drones act as aerial base stations to assist the base station in providing services to users. They hover above the city. It is assumed that each drone is deployed in a sub-region to provide edge storage services to vehicles, with no overlap between sub-regions. At time t, the position of the drone is $H^t_u = (x^t_u, y^t_u, h)$, where (x^t_n, y^t_n) denotes the horizontal projection of the drone on the ground, and h is the height of the drone, assuming all drones fly at a fixed height. The position of the vehicle is denoted by $H^t_v = (x^t_v, y^t_v, 0)$.

Fig. 2 illustrates the four clearer content delivery paths. Vehicles act as both content requesters and providers. If the requested content exists in the vehicle's local cache, the request will be completed immediately without the need to establish a communication link. Otherwise, the vehicle can retrieve the content through one of four paths:

V2V Link: Retrieve content from nearby vehicles, as shown in Process 1.

V2U Link: Retrieve content from a UAV above its active area, as shown in Process 2.

V2U' Link**:** Cooperate with neighboring UAVs to retrieve content, as shown in Process 3.

V2B Link**:** Retrieve content from the base station, as shown in Process 4.

B. Cache Model

In this section, we will discuss the hierarchical caching decision and replacement decision models designed for different users.

1) Content Request: The frequency distribution of each content being requested by users in the file library is denoted as $p = \{p_1, p_2, \cdots p_F\}$, where p_f is the request probability

of content f, following a Zipf distribution. For any content f, the global popularity in the area and the probability of a user requesting the file are given by

$$p_f = \frac{f^{-\gamma}}{\sum_{j=1}^{F} j^{-\gamma}} \tag{1}$$

Here, γ is the Zipf exponent, indicating the skewness of the popularity distribution. The larger the γ, the more concentrated the popular content is; conversely, the smaller the γ, the more evenly distributed the popular content is.

2) Caching Placement Decision: φ_{v_m,f_k} represents the interest preferences of the vehicle user, which are closely related to the content's topic characteristics. Suppose there are j topics for the content, and the topic set is denoted as $W = \{w_1, w_2 \ldots . w_J\}$. Define $e_k(f_k, w_j)$ as a binary variable indicating the relevance between content f_k and topic w_j, equals 1 if they are related, and 0 if they are not.

Based on the vehicle's historical request information, the caching value of the currently requested content is determined—whether it is a fleeting interest or driven by content preference. The average topic similarity between the K historically requested contents by the vehicle v_m is denoted by $Q(f, f')$, defined using the average cosine similarity [7].

$$Q(f, f') = \frac{2}{K(K-1)} \sum_{f,f'} \frac{\sum_{j=1}^{J} e(f, w_j) e(f', w_j)}{\sqrt{\sum_{j=1}^{J} e(f, w_j)^2 e(f', w_j)^2}} \tag{2}$$

The relevance between the currently requested content f_k by the vehicle v_m and the historical average request information $Q(f, f')$ is determined by the topic to which the requested content belongs. This relevance is used to represent the vehicle user's preference level for the current requested content f_k

$$\varphi_{v_m,f_k} = \frac{\sum_{j=1}^{J} e(f_k, w_j) Q(f, f')}{\sqrt{\sum_{j=1}^{J} \left[e(f_k, w_j)\right]^2} \sqrt{\sum_{j=1}^{J} \left[Q(f, f')\right]^2}} \tag{3}$$

3) Caching Replacement Decision: Based on the above caching strategy, the caching status in the cache list of each node is represented by the matrix C as follows

$$C = \begin{pmatrix} c_{1f_1} & \cdots & c_{1f_n} \\ \vdots & \ddots & \vdots \\ c_{Mf_1} & \cdots & c_{Mf_n} \end{pmatrix} \tag{4}$$

To simplify the notation, let M represent the total set of all vehicles and UAVs. The binary decision variable $c_{M,f}$ is used to indicate the caching status of content f by vehicles and UAVs, where $c_{M,f} = 1$ if content f is cached, and $c_{M,f} = 0$ otherwise. Considering the limited caching resources and assuming that each piece of content is cached in only one location, along with the cache size constraints, we have the following

$$\mathrm{C}_u, \mathrm{C}_v \le C_M, \forall v, u \tag{5}$$

C_u represent the caching capacity of UAVs, C_v represent the caching capacity of vehicles, and C_M represent the maximum caching resource limit. In each time slot, edge nodes can either add newly arrived request content to their cache or replace/update old content. We use C_M^{t-1} to denote the caching capacity status of cache node M at time slot $t-1$, and c_M^t and c_M^{t-1} to denote the caching status of content f at cache node M at time slots t and $t-1$, respectively. s_f represent the size of the requested content. When the caching capacity is insufficient, a replacement mechanism is triggered

$$\mathrm{C}_M^{t-1} + \sum_{f=1}^{F} (c_M^t - c_M^{t-1}) \times s_f > \mathrm{C}_M \tag{6}$$

Due to limited cache space, in order to maximize the utility of caching space, and by combining global and individual popularity, the caching value of content f_k at different levels of nodes is defined as

$$V_f^t = \omega_1 \varphi_{v_m,f_k} + \omega_2 p_f \tag{7}$$

Here, ω is an adjustment parameter used to tailor different caching decision schemes based on the actual needs and scenarios of vehicles and UAVs.

Where the caching replacement decision defined as: $x_{f_k}^t = e^{V_f^t - \mu s_f^t}$. Let V_f^t represent the caching value at the current time t, and let s_f^t represent the size of the requested content. Based on $x_{f_k}^t$, the cached content is sorted.

$$c_{Mf_k}^t = \begin{cases} 1, & \text{if } x_{f_k}^t > x_{f_{k'}}^{t-1} \\ 0, & otherwise \end{cases} \tag{8}$$

Algorithm 1 A Hierarchical Collaborative Cache Replacement Algorithm Based on User Preferences

Initialization:
Getting vehicle ID information
Read data from the dataset: vehicle historical request information
Calculate the average similarity of the requested content based on the user history by Eq. (2)
return vehicle user preferences
Loop:
When a user request arrives
If the requested content is already in the cache
Perform a cache hit
return the cached content list
else:
Calculate the value V of the content accoding Eq.(3) Eq.(7)
If the cache space allowance is calculated based on Equation (6)
cache it
else:
Decide whether to replace existing content in the cache using Eq. (7) Eq. (8)
Cache the new content in the available cache space
Return the updated cached content list
End Loop

When the caching value of content f_k is higher than that of the currently cached content f_k', the replacement action is triggered, represented by $c_{Mf_k}^t$. Algorithm 1 describes our

proposed method for replacing the requested content and the one with the lowest value. If the content to be replaced exceeds the available storage capacity, the algorithm triggers the replacement of the second-to-last content until sufficient cache space is available. Additionally, in practice, users may have demands for the timeliness and freshness of information. However, exploring more realistic and dynamic content version update rate models is not the focus of this research and will not be elaborated upon here.

C. Incentive Model

Due to the consumption of resources and privacy concerns associated with content sharing among vehicles, which can lead to selfish behavior and reluctance to share content, an incentive-based collaborative content sharing scheme has been designed to encourage vehicles to actively participate in content sharing.

Assume that the social contribution rate of node i is SC_i. It is defined as the ratio of effectively contributed content to the amount of cache space used, representing the social and economic benefits generated by the resources occupied by the social cache nodes. When Vehicle v_m^r successfully shares content with another vehicle, its corresponding contribution rate increases

$$SC_a = \frac{s_f p_f}{\sum_{f=1}^{F} c_{M,f}^t s_f} \tag{9}$$

Let h_{v2v} represent the decision of content sharing between vehicles. Taking into account the amount of content actually shared by the vehicles, the average social contribution rate of each vehicle can be calculated using

$$SC_i(t) = \frac{1}{T}\sum_{t=1}^{T} \frac{h_{v2v} s_f p_f}{\sum_{f=1}^{F} c_{M,f}^t s_f} \tag{10}$$

When cache nodes receive multiple user requests simultaneously, UAVs will prioritize and queue the requests based on the contribution value of each cache holder, giving priority to users with higher social contribution rates.

At the same time, social caching resources will be tilted in their favor, with the proportion of edge node cache space allocated to content related to their interests increasing accordingly. This is specifically reflected by incorporating the social contribution rate into the caching value Eq. (4)

$$V_f^t = \omega_1 \varphi_{v_m, f_k} + \omega_2 (p_f + \alpha SC_i) \tag{11}$$

Here, α is an adjustment factor for social contribution participation, incentivizing vehicle users to provide better service to others in order to receive better service themselves. Additionally, considering the impact of delay tolerance and packet loss rate, it is assumed that vehicle-to-vehicle content transmission does not exceed two hops.

D. Communication Model

1) Transmission Rate Between Vehicles (V2V): According to Shannon's theory, the transmission rate for sharing content between vehicles is given by

$$\mathrm{R}_{vv'}(t) = B_{vv'} \log_2 \left(1 + \frac{P_{vv'} g_{vv'}(t)}{N_0 B_{vv'}}\right) \tag{12}$$

$B_{vv'}$ is the transmission bandwidth between vehicles. $P_{vv'}$ is the transmission power of the vehicles. N_0 is the noise power. $g_{v,v'}(t) = \frac{g_0}{l_{vv'}^2(t)}$ is the channel gain between vehicles. $l_{vv'}^2(t)$ is the distance between vehicles. g_0 represents the power gain per unit reference distance.

A communication link is established when the communication channel's SNR exceeds the threshold η_{veh}

$$h_{v,v'}^f = \begin{cases} 1, & \text{if } \gamma_{v,v'} > \eta_{veh} \\ 0, & otherwise \end{cases} \tag{13}$$

2) Transmission Rate From Vehicles to UAVs (V2U): The distance between the drone and the vehicle at time t, based on their position coordinates, can be represented as

$$l_{u,v}(t) = \|H_u^t(t) - H_v^t(t)\|_2 \tag{14}$$

$H_v^t(t)$ are the coordinates of the vehicle at time. $H_u^t(t)$ are the coordinates of the UAV at time. According to the literature [3], the line-of-sight channel gain between a vehicle and UAV n can be represented using the free-space path loss model as: $h_{v,u}(t) = \frac{g_0}{(l_{u,v}(t))^2}$. Thus, the transmission rate is given by

$$R_{u,v} = B_{u,v} \log_2 \left(1 + \frac{P_{vu}{}^t h_{v,u}(t)}{\sigma^2}\right) \tag{15}$$

Let $B_{\mathrm{u,v}}$ represent the bandwidth allocated by the UAV to vehicle users, which is distributed equally among the associated users. $P_{\mathrm{u,v}}^t$ denotes the transmission power of UAV n to user m at time interval t, and σ^2 represents the variance of Gaussian noise.

3) Transmission Rate Between UAVs (U2U): The transmission rate for sharing content between UAVs, based on Shannon's theory, is given by

$$R_{u,u'} = B_{u,u'} \log_2 \left(1 + \frac{P_{u,u'} \mathrm{g}_0}{\sigma^2 \|H_u^t(t) - H_{u'}^t(t)\|_2}\right) \tag{16}$$

To ensure that the vehicle successfully receives the requested content, the minimum transmission power of the UAV should satisfy the following condition

$$P_{\min}^{t,v,u} = \frac{\gamma_{v,u}^t \sigma^2}{h_{v,u}(t)} = \frac{(2^{R_{vu}/B_{u,v}} - 1)\sigma^2}{h_{v,u}(t)} \tag{17}$$

Let $h_{u,u'}^f$ represent the shared decision variable between UAV u and UAV u'. A communication link is established when the signal-to-noise ratio (SNR) between the UAVs exceeds the threshold η_{UAV}.

$$h_{u,u'}^f = \begin{cases} 1, & \text{if } \gamma_{u,u'} > \eta_{UAV} \\ 0, & otherwise \end{cases} \tag{18}$$

4) Transmission Rate From Vehicle-to-Base Station (V2B): The communication from the base station to vehicle users can be carried out through the V2B link, with the transmission rate given as follows

$$R_{bv}^t = B_{bv}^t \log_2 \left(1 + \frac{P_{v,b} g_{v,b}(t)}{N_0 B_{b,v}}\right) \tag{19}$$

IV. Problem Formation

In this section, we design an optimization problem with the objective of maximizing cache benefits while improving network quality of service. This is achieved by finding the optimal content replacement strategy under the constraints of limited UAV power and finite cache node resources.

A. Transmission Delay

1) The Delay in Obtaining Content Through the V2V Link: Vehicles broadcast requests, and if the requested file is found among neighboring vehicles within their communication range, a V2V communication link can be established to obtain the content. Considering that channel gain varies with time [13], the transmission delay $d^f_{v',v}$ for obtaining content f of size s^t_f can be expressed as

$$\int_0^{d^f_{v',v}} R_{vv'}(t)dt = s^t_f \tag{20}$$

2) The Delay in Obtaining Content Through the V2U Link: UAVs provide service to vehicles through a single-channel system. If the channel is not occupied, the content is transmitted to the vehicle immediately. Otherwise, the request waits in a queue.

a) Requested Content Is in the UAV Cache List: Considering the maximum tolerable delay of the task and vehicle movement, the UAV calculates the task waiting time requested by the vehicle. The delay for the task to be transmitted from the UAV to the vehicle is represented as

$$d_{u,v} = \sum_{f=1}^{F} \frac{{s_f}^{t-1}}{\bar{R}_{u2v}} + \frac{s^t_f}{\bar{R}_{u2v}} \tag{21}$$

To simplify the expression for transmission time in Eq. (20), we approximate the problem by taking the average transmission rate over the time period, denoted as $\bar{R}(t)$. The first part represents the delay of content waiting in the UAV's processing queue, while the second part represents the delay of the UAV processing the current content request.

b) Requested Content Is Not in the UAV Cache List: If the requested content is in the cache list of UAV u' within communication range, and UAV u' transmits the content to u through content sharing, UAV u acts as a relay to forward it. The transmission delay is represented as

$$d^f_{u,u'} = \sum_{f=1}^{F} \frac{{s_f}^{t-1}}{\bar{R}_{u2v}} + \frac{s^t_f}{\bar{R}_{u2u}} + \frac{s^t_f}{\bar{R}_{u2v}} \tag{22}$$

Let binary variables k_1 and k_2 represent whether the content transmission delay meets the requirements, and t_{v_m} represent the average dwell time of vehicle v in the area. This value can be obtained by analyzing the historical dwell data of all vehicles in the area. δ_f represents the maximum tolerable delay for content f.

$$k_1 = \begin{cases} 1, & \text{if } d_{u,v} \le \min\left\{t_{v_m}, \delta_f\right\} \\ 0, & otherwise \end{cases} \tag{23}$$

$$k_2 = \begin{cases} 1, & \text{if } d_{u',v} \le \min\left\{t_{v_m}, \delta_f\right\} \\ 0, & otherwise \end{cases} \tag{24}$$

If $k_1 = 1$, the request from vehicle v will be served by UAV u; otherwise, the content service will be obtained through the V2B link. Similarly, if $k_2 = 1$, the content request task will be sent by a neighboring UAV; otherwise, the content will be retrieved through the base station. When users in the region request content simultaneously, service will be provided in sequence based on the SC_i value

3) The Delay in Obtaining Content Through the V2B Link:

$$\int_0^{d^f_{b,v}} R_{vb}(t)dt = s^t_f \tag{25}$$

In summary, the total time for the user to receive content f through four types of links is

$$D^{total}_{f,v} = \begin{bmatrix} c^t_{vf} d^f_{v',v} \\ +(1-c^t_{vf})k_1 c^t_{uf} d^f_{u,v} \\ +(1-c^t_{vf})(1-k_1 c^t_{uf})k_2 c^t_{u'f} d^f_{u,u'} \\ +(1-c^t_{vf})(1-k_1 c^t_{uf})(1-k_2 c^t_{u'f}) d^f_{b,v} \end{bmatrix} \tag{26}$$

c^t_{vf} represents whether the requested content is cached in nearby vehicles, $k_1 c^t_{uf} = 1$ indicates that the content requested by vehicle v is cached in the UAV and does not exceed the waiting time,similarly, $k_2 c^t_{u'f} = 1$ indicates that content f is cached in a neighboring UAV. $d^f_{v',v}, d^f_{u,v}, d^f_{u,u'}, d^f_{b,v}$ represent the transmission delays for the content to be served to the user by the vehicle, UAV, neighboring UAV, and base station, respectively.

B. System Energy Cost

1) *The V2V Energy Consumption Can Be Expressed as:*

$$E^f_{v,v'} = P_{v,v'} d^f_{v',v} \tag{27}$$

2) *The V2U Energy Consumption Can Be Expressed as:*

a) *The U2U energy consumption:* The transmission delay energy consumption for UAV u above the vehicle to transmit content f to vehicle v is

$$E^f_{v,u} = P_u d_{v,u} = P_u \left(s_f / \overline{R_{u,v}}\right) \tag{28}$$

b) *The U2U' energy consumption:* The transmission energy consumption for the neighboring UAV u' above the vehicle to transmit content f to vehicle v can be expressed as

$$E^f_{v,u'} = E^f_{v,u} + E^f_{u',u} = P_u \left(s_f / \overline{R_{u,v}} + s_f / \overline{R_{u,u'}}\right) \tag{29}$$

Therefore, the transmission energy consumption of UAV u can be expressed as

$$E_{tra} = E^f_{v,u} + \sum\nolimits_u E^f_{u,u'} \tag{30}$$

3) *The V2B Energy Consumption Can Be Expressed as:*

$$E_{bv} = P_{bv} d^f_{b,v} \tag{31}$$

In summary, considering the transmission cost, the system energy cost for vehicle v to receive content f through

the four types of links at each time slot t can be expressed as

$$S_{f,v}^{total} = \begin{bmatrix} c_{vf}^t \varpi_v E_{v,v'}^f + \\ (1-c_{vf}^t) k_1 c_{uf}^t \varpi_u E_{v,u}^f + \\ (1-c_{vf}^t)(1-k_1 c_{uf}^t) k_2 c_{u'f}^t \varpi_u E_{v,u'}^f + \\ (1-c_{vf}^t)(1-k_1 c_{uf}^t)(1-k_2 c_{u'f}^t) \varpi_b E_{bv}^f \end{bmatrix} \tag{32}$$

The costs $\varpi_v, \varpi_u, \varpi_b$ represent the unit energy consumption costs for transmitting content through different links, based on factors such as transmission distance and other influencing factors $\varpi_b > \varpi_u > \varpi_v$.

C. Problem Formation

System delay and energy consumption are two key components of system cost. Since there is a significant disparity between delay and energy consumption, we express the time benefit in the following normalized form

$$U_t = \frac{d_{b,v}^f - D_{f,v}^{total}}{d_{b,v}^f} \tag{33}$$

$d_{b,v}^f - D_{f,v}^{total}$ represents the time improvement gained by retrieving content from an edge node compared to getting it from the base station. Here, normalization has been applied.

Similarly, the energy cost benefit function can be expressed as

$$U_e = \frac{\varpi_b E_{bv}^f - S_{f,v}^{total}}{\varpi_b E_{bv}^f} \tag{34}$$

To achieve both improved QoS and controlled energy consumption, aiming for efficient resource utilization and network performance optimization, we define the caching benefit function as $U = (U_t + U_e)p_f$, where p_f represents the probability of file requests. Ultimately, our optimization problem can be formulated as minimizing system delay and energy consumption while maximizing system benefits

$$P: \max_{c_{vf}^t, c_{uf}^t} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \sum_{f=1}^{F} \sum_{v=1}^{m} \sum_{u=1}^{n} U \tag{35}$$

$$\begin{aligned} s.t. \quad & C_1: c_{vf}^t \in \{0,1\}, \quad \forall f, v \\ & C_2: c_{uf}^t \in \{0,1\}, \quad \forall f, u \\ & C_3: \sum_{f=1}^{F} s_f c_{Mf}^t \le C_{\mathrm{M}}, \quad \forall f \in F, M = V \cup U \\ & C_4: \sum_{f=1}^{F} c_{Mf}^t \le 1, \quad \forall f \in F \\ & \mathrm{C}_5: D \le \delta_f \end{aligned} \tag{36}$$

C_1C_2 represents binary decision variables, C_3 indicates that the cache sizes of vehicles and UAVs do not exceed their capacity, C_4 ensures that content can only be cached in one place, and C_5 indicates that the maximum tolerable delay is not exceeded.

V. Problem Analysis And Resolution

Given the complexity and strong dynamics of problem P, we address this issue using two methods and validate them against each other. For the theoretical solution, we use the DCA algorithm with a penalty function to solve the cache placement problem. For the simulation solution, we validate by combining the caching algorithm with the D3QN network.

A. Theoretical Solutions

Since P have non-convex constraints. By mapping P to a multiple knapsack problem (MKP), it can be shown that this problem is NP-hard (Lemma 1).The solution space of this problem grows exponentially and cannot be solved by fast algorithms in polynomial time.

For NP-hard problems like the multi-knapsack problem, heuristic algorithms and dynamic programming are commonly used to solve them, transforming them into a DCP convex optimization problem is uncommon. These methods may sacrifice some accuracy but can typically find near-optimal solutions within a reasonable time frame.

Given that the objective function is a linear combination of two terms, and assuming that within each time slot in the given environment, the movement of vehicles and drones is minimal, the cache placement problems c_{uf}^t, c_{vf}^t can be simplified as follows

$$P_1: \max_{c_{vf}^t, c_{uf}^t} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \sum_{f=1}^{F} \sum_{v=1}^{M} \sum_{u=1}^{n} U \tag{37}$$

$$\begin{aligned} s.t. \quad & C_1: c_{uf}^t, c_{vf}^t \in \{0,1\}, \quad \forall f, v, u \\ & C_2: \sum_{f=1}^{F} s_f c_{mf}^t \le C_{\mathrm{M}}, \quad \forall m \in M, \\ & \forall f \in F, M = V \cup U \\ & C_3: \sum_{f=1}^{F} c_{mf}^t \le 1, \forall m \in M, \quad \forall f \in F \end{aligned} \tag{38}$$

Due to the binary nature of the constraints C_1, the problem P_1 is non-convex. Therefore, we first relax the binary cache placement variables to continuous variables. P_1 can be rewritten as $P_{1.1}$

$$P_{1.1}: \max_{c_{vf}^t, c_{uf}^t} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \sum_{f=1}^{F} \sum_{v=1}^{M} \sum_{u=1}^{n} U \tag{39}$$

$$\begin{aligned} s.t. \quad & C_1': 0 \le c_{uf}^t, c_{vf}^t \le 1, \quad \forall f \\ & C_2 - C_3 \end{aligned} \tag{40}$$

Since the objective function and constraints are linear, the problem $P_{1.1}$ is convex. Therefore, it can be solved using standard convex optimization methods. However, because the cache placement variables have been relaxed to continuous values between 0 and 1, it cannot be guaranteed that the cache decision variable c_{mf}^t will converge to 0 or 1. To address this, a penalty function $\mathrm{P}(c) \triangleq c(c-1)$, which is a convex

function [17], is introduced. Consequently, the subproblem $P_{1.1}$ is reformulated as

$$P_{1.2}: \max_{c_{vf}^t, c_{uf}^t} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \sum_{f=1}^{F} \sum_{v=1}^{M} \sum_{u=1}^{n} U + \kappa \mathrm{P}(c)$$
$$s.t. \quad C_1', C_2, C_3 \tag{41}$$

Let the penalty factor $\kappa > 0$. If the objective in $P_{1.2}$ is transformed into $f(c) \triangleq U - (-\kappa \mathrm{P}(c))$, the original problem can be considered as a difference of convex functions (DC function), where a convex function (1/log) is subtracted from a concave function under convex constraints. To transform $P_{1.2}$ into a convex problem, replace the penalty function $\mathrm{P}(c)$ with the first-order Taylor expansion at the (n+1)th iteration

$$P'(c) \triangleq \kappa(P(c_{mf}^t)^n + \nabla P(c_{mf}^t)^n (c_{mf}^t - (c_{mf}^t)^n) \tag{42}$$

where

$$\nabla P(c_{mf}^t)^n = 2(c_{mf}^t)^n - 1 \tag{43}$$

Therebefore,we have

$$P'(c) \triangleq \kappa(c_{mf}^t(2(c_{mf}^t)^n - 1) - (c_{mf}^t)^{2n}) \tag{44}$$

Therefore, the subproblem $P_{1.2}$ can be approximately transformed into the following linear programming problem

$$P_{1.3}: \max_{c_{vf}^t, c_{uf}^t} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \sum_{f=1}^{F} \sum_{v=1}^{M} \sum_{u=1}^{n} U + \kappa P'(c) \tag{45}$$
$$s.t. \quad C_1', C_2, C_3 \tag{46}$$

Algorithm 2 Proposed Iterative Algorithm to Solve P1

Initialization: Set n = 0 and initialize c convergence threshold ε.
Input: system parameters, constraints, cache capacity, content size, number of contents, tolerated latency, objective function
Output: The optimal cached decision variables c and its corresponding caching benefit
repeat
Compute the gradient of the penalty function $\nabla P(c^n)$.
Update the cache placement variables c
Check the convergence of $\|c_{new} - c\| < \varepsilon$, then exit the loop.
until Convergence

The optimal cache decision at a specific moment can be obtained by solving the linear approximation of problem P, denoted as problem P1.3. This solution represents the optimal configuration for that particular moment and is a one-time static solution. Subproblem $P_{1.3}$ is a linear programming problem, and its solution can be obtained using a standard optimization solver, such as CVXPY, as described in Algorithm 2.

If the resulting cache placement variables c can converge to binary values, then the relaxation is tight, and the constraints C_1' and C_1 are equivalent. This means that $P_{1.3}$ achieves the optimal solution, and the solution of $P_{1.3}$ is also a feasible solution for problem P_1. By applying the lower bound results from Eq. (41), we can obtain a lower bound, which is the optimal solution of the linear programming problem [16]. Due to the use of approximation methods, this lower bound might not be the optimal solution to the original problem, but it still provides a feasible solution. The solution of $P_{1.3}$ also satisfies P_1^c, indicating that at least a local optimum is achieved. Theoretically, with a sufficiently large κ, $\kappa \mathrm{P}(c)$ would still be zero when the optimal solution is obtained. However, in practical computations, numerical tolerance may exist. If $\exists \varepsilon, \varepsilon > 0, \forall \kappa, \kappa \mathrm{P}'(c) < \varepsilon$ holds (where ε is sufficiently small and κ is sufficiently large), then the obtained solution is reasonable.

B. A Solution Based on D3QN Reinforcement Learning

Since the theoretical solution above is derived under fixed variables in a specific environment, it is the "optimal" configuration for a specific moment, a one-time static solution. However, in real-world environments, many uncontrollable factors can affect the situation. Therefore, it is considered to use deep reinforcement learning to train agents to learn from the highly dynamic environment and make action decisions to address the above issues.

1) D3QN Algorithm Model: D3QN improves the accuracy of Q-value estimation and the effectiveness of the policy through the use of Double Q-networks and a Dueling network structure. The network structure includes an input layer for state representation, hidden layers for feature extraction, branch layers, and an output layer for estimating Q-values.

In Q-Learning, by maximizing the Q value, the agent can choose the optimal action. The standard Q-Learning iteration formula is as follows

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)) \tag{47}$$

$Q(s_t, a_t)$ is the current state s_t to perform the action a_t Estimated Q value. r_{t+1} It is a time step t instant rewards obtained. γ is a discount factor. $\max_{a'} Q(s_{t+1}, a')$ is the next state s_{t+1} Estimation of the maximum Q value in.

Double Q-Learning [4] reduces bias by calculating the Q value independently. The update formula for Double Q-Learning is

$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha(r_{t+1} + \gamma Q_2(s_{t+1}, \arg\max_{a'} Q_1(s_{t+1}, a')) - Q_1(s_t, a_t)) \tag{48}$$

$$Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha(r_{t+1} + \gamma Q_1(s_{t+1}, \arg\max_{a'} Q_2(s_{t+1}, a')) - Q_2(s_t, a_t)) \tag{49}$$

Q_1 and Q_2 two separate Q-value estimates are described. When selecting an action, one Q function is used to select the action and the other Q function is used to calculate the reward and expected future return, thus reducing the overestimation of the Q value.

The Dueling Network is an improved neural network structure that is used to reduce variance in Q-value estimates. It breaks down the Q function into two parts:The state value

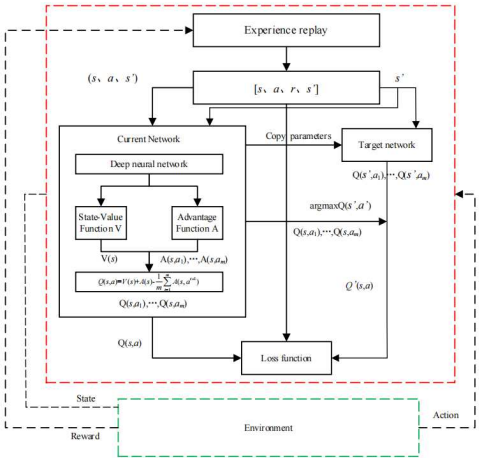


Fig. 3. The neural network structure of proposed algorithm.

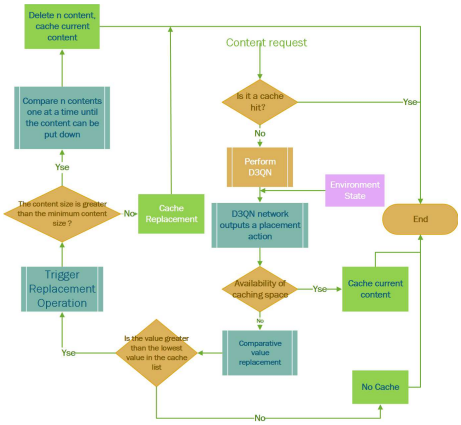


Fig. 4. The flowchart of the PC-ID3QN algorithm.

function $V(s)$ and Dominance function $A(s,a)$ and synthesize the Q value in the following ways [18]

$$Q(s,a) = V(s) + \left(A(s,a) - \frac{1}{|A(s)|}\sum_{a'} A(s,a')\right) \tag{50}$$

V(s) Represents the state s value. $A(s,a)$ Represents in the state s to perform the action a advantages. $\frac{1}{|A(s)|}\sum_{a'} A(s,a')$ is a normalization term that adjusts the advantage function so that its output can be combined with state values.

The D3QN select action Pass *argmax* to select the action corresponding to the maximum Q value

$$a_t = \arg\max_a \left(V(s_t) + A(s_t,a) - \frac{1}{|A(s)|}\sum_{a'} A(s_t,a')\right) \tag{51}$$

The only difference between D3QN and Dueling DQN algorithms is the method used to compute the target value. The method for computing the target value in D3QN can be expressed as

$$Q_{t\arg et} = R_{t+1} + \gamma \max_A Q(S_{t+1}, A, \theta_t^-) \tag{52}$$

The network structure and update method of D3QN [10] are as follows

$$Q(s_t,a_t)R_t + \gamma\left(V(s) + \left(A\left(s_{t+1}, \arg\max_a Q\left(s_{t+1},a\right)\right)\right.\right.$$
$$\left.\left. - \frac{1}{|A|}\sum_{a'} A\left(s_{t+1},a_{t+1}\right)\right)\right) \tag{53}$$

Overall, compared to DQN, DDQN, and Dueling DQN, D3QN can achieve more stable and efficient performance across a wide range of tasks, making it suitable for real-world problems. Therefore, we use D3QN as a model-free DRL model to propose our content importance evaluation model.

2) Caching Algorithm Based on PC-ID3QN: Given the complexity of the environment, to achieve this goal, we model the aforementioned optimization problem as a Markov Decision Process (MDP) for solving. Fig. 3 shows the neural network structure of the proposed algorithm, which combines the caching environment with D3QN. The base station, as the agent, observes the surrounding environment state and decides whether to cache content at the caching node. The agent then receives a set of rewards, which are stored in the experience replay buffer. Fig. 4 shows the structure of the proposed PC-ID3QN algorithm. At each iteration, the agent interacts with the environment in two ways: If the requested content is cached at the edge node, the agent immediately receives the maximum reward, bypassing the caching model and the PC-ID3QN algorithm; otherwise, the agent applies the caching model and the PC-ID3QN-based algorithm to optimize the caching decision.

***State Space*:** S In this scenario, the agent needs to cache tasks based on the quantity and size of the requested content, caching resources, and external environmental information. Therefore, the system state S at time slot t includes the following information

$$\mathcal{S} = \{H_v^t, \vartheta, \omega_\theta, \rho_v, c_{Mf_n}, \mathrm{C}_M, \Theta_m(t), V_f^t\} \tag{54}$$

- H_v^t: Position of the vehicle making the request at time slot t;
- ϑ: At time slot t, the information regarding the list of requested content f;
- ω_θ: Vehicle Movement Angle: It is calculated based on the previous and next moments;
- ρ_v:Traffic Density: The number of surrounding vehicles;
- c_{Mf_n}: Cache Status:Indicates whether the cache node k is caching the content f at time slot t;
- C_M: The available storage capacity of the device or cache node at time slot t;
- $\Theta_m(t)$: The social contribution of the vehicle requesting the content at time slot t;
- V_f^t: The index or importance of the content f in the cache at time slot t;

***Action Space*:** In the proposed model, the agent's actions determine whether to cache the content, where to cache it. Thus, the action space includes the following elements: $\left\{c_{vf}^t, c_{v'f}^t, c_{uf}^t, c_{u'f}^t\right\}$ no caching, caching to a vehicle, caching to a neighboring vehicle, caching to a UAV, and caching to a neighboring UAV.

Therefore, the action $a_t \in \mathcal{A}$ at time slot t can be represented as

$$a_t = \begin{bmatrix} c_{v1}^t \ldots c_{vf}^t \ldots c_{v'1}^t \ldots c_{v'f}^t \\ c_{u1}^t \ldots c_{uf}^t \ldots c_{u'1}^t \ldots c_{u'f}^t \end{bmatrix} \tag{55}$$

Algorithm 3 Cooperative Layered Caching Based on ID3QN

Initialization: Initialize Q-network $Q(s,a)$ and target network $Q'(s,a)$
Input: Max train iteration G; discount factor; initial tuple (a_t, s_t, r_t, s_t+1); mini-batch size H; learning rate lr_n, parameters θ and θ' and experience replay buffer D
Output: The action of each agent, i.e., the cache strategy.
for each episode **do**:
  Reset environment, obtain the original state S.
  for time step = 1, 2,..., T **do**:
    if $c_{Mf_n}=1$:
      obtain next state by S and reward by Eq. (54) and Eq. (57)
      Update state $S_t = S_{t+1}$
    else
      Input the S_t to the Q-network Eq.(53), choose the action Eq. (51);obtain the next state S_{t+1} and reward by Eq. (54) and Eq. (57); store the transition (a_t, s_t, r_t, s_t+1) into replay memory D; Update Q-network(the actor network and critic network) using replay buffer D, θ', S_{t+1} by Equation (53);
    end
  end
  calculate average revenue for the episode.
end

Cost Function: The reward function $R_i(t)$ contains two components

$$R_i(t) = R(t) + \alpha SC_i(t) \tag{56}$$

Let $R(t)$ represent the system's caching utility objective function. Additionally, let $SC_i(t)$ be the social contribution rate, and α be the parameter that adjusts the incentive for the social contribution rate. We use a piecewise function to represent the reward based on the actual delivery delay compared to the maximum tolerable delay. If the actual delivery delay of the content is less than its maximum tolerable delay, the reward is given by the system utility as described in Eq. (33). If the actual delivery delay exceeds the maximum tolerable delay, the reward is the negative value of the system utility.

Thus, the reward function can be expressed as

$$R(t) = \begin{cases} U, & \text{if} D_t < \delta_f \\ -U, & \text{if} D_t > \delta_f \\ \max\_reward, & \text{if} \varsigma = 1 \end{cases} \tag{57}$$

***Transition probability*:** $P(s,s')$ can represent the transition probability from the state s at time slot t to the state s' at time slot $t+1$. This transition probability can be expressed as

$$P(s,s') = P_r\{s_{t+1} = s' | s, \mathcal{A}\} \tag{58}$$

C. Complexity Analysis

In the D3QN algorithm with the introduced incentive mechanism, the time complexity mainly depends on the number of episodes (E) and the time steps (T). The time complexity is also proportional to the network size (N). The time complexity related to network size can be expressed as $O(N) = 2 \cdot \left(\sum_{l=0}^{L_{fw}} n_l^i \cdot n_l^o\right) + \left(\sum_{l=0}^{L_{bw}} n_l^i \cdot n_l^o\right)$. Here, L_{fw} and L_{bw} represents the number of layers for both forward and backward propagation, where n_l^i and n_l^o are the number of input and output neurons in layer l, respectively. Additionally, the time complexity for sampling from the experience replay buffer is $O(B)$, where B is the batch size. After introducing the incentive mechanism, the calculation of social contribution rates typically depends on the user's history of behavior and contribution. Assuming each user's history record is h_i, the time complexity for incentive calculation is $\mathcal{O}(u \cdot h_i)$, where u is the number of users. The time complexity for calculating user preferences and caching value depends on the number of contents m and the user's level L_i, which is $\mathcal{O}(m \cdot L_i)$. During the cache strategy adjustment, recalculating cache value and selecting or replacing content has a complexity of $\mathcal{O}(m)$. Therefore, the overall time complexity of the algorithm can be roughly expressed as $\mathcal{O}(E \cdot T \cdot (N + B + u \cdot h_i + m \cdot L_i + m))$.

The space complexity mainly stems from the storage of neural network parameters, the experience replay buffer, and the storage requirements for user contribution rates, incentives, and cache states. Since there are two networks (the Q-network and the target network), the parameter space complexity is $\mathcal{O}(2W)$, where W is the number of weights in a single network. Additionally, the space complexity of the experience replay buffer and the cache is $\mathcal{O}(R)$ and $\mathcal{O}(C)$, respectively, where R and C represent the sizes of the experience replay buffer and cache.The incentive mechanism may introduce additional space requirements: each user's preference vector P_i, level L_i, and social contribution rate SC_i need to be stored, with a space complexity of $\mathcal{O}(U)$. The space complexity for storing each user's preferences for m contents is $\mathcal{O}(M)$. Therefore, the total space complexity of the algorithm is $\mathcal{O}(2W + R + C + U + M)$.

VI. Simulation Results and Analysis

A. Simulation Parameters

In this section, we validate the proposed algorithm's performance in an IoV environment through simulations involving. In specific terms, the operating system we use is Windows 10, 64-bit, the CPU is i7-8565U, 1.8GHZ, with 8GB RAM. The simulation tool we use is PyCharm and the programming language used is Python, where the Python version is 3.8, Pytorch version 1.8, Anaconda3(64-bit), pip version 23.2, Pandas version 1.4.4, Numpy version 1.23.5. Additionally, without losing generality, we set the transmission power of vehicles to 20 dBm, UAVs to 30 dBm, and the MBS to 35 dBm, with Gaussian white noise set to −95 dBm. The V2V communication has a bandwidth of 20 MHz and a carrier frequency of 5.9 GHz, while the V2U communication has a bandwidth of 160 MHz and a carrier frequency of 5 GHz. Table II details the simulation system parameters.

User preference data was sourced from the Movie Lens 1M dataset, selecting ratings from the top 30 users on 20 pieces of content, resulting in 600 files. We then extracted 100 pieces of content for validation. Note: In this case, the probability of requesting files follows a Zipf distribution, consistent with the request model mentioned above.

To highlight the advantages of the algorithm, we used the same parameters for Dueling DQN, Double DQN, and D3QN: the learning rate γ, which affects the convergence speed of

TABLE II
SIMULATION PARAMETERS

Parameter	Value
Number of content requests per time slot F	30, **50**,100,150
Number of UAVs n	3[36]
Number of vehicles m	10, **30**, 50,70
Vehicle speed v	120km/h
UAV cache capacity C_u	300MB[25][36]
Vehicle cache capacity C_v	50MB
Energy unit price	0.00658
Vehicle request content tolerance delay	0.2s
Content size s_f	8, **15**, 20,25MB
Content Topic Type W	18
Vehicle Communication Range R	50m
UAV altitude H	100m[25]
UAV coverage radius	100m[36][21]
V2V transmission power P	20dBm[25][36]
V2U transmission power P	30dBm[25][36]
V2B transmission power P	35dBm[25][36]
Background noise power	-95 dBm[25][36]
SNR threshold	30 dB[35]
Simulation of site size	400m×400m[5]
V2V bandwidth and carrier frequency	20 MHz 5.9 GHz
V2U bandwidth and carrier frequency	160 MHz 5 GHz[35]
V2B bandwidth and carrier frequency	60 MHz 3.5 GHz

the entire network, is set to 0.001; the experience replay buffer is 3000; the number of iterations is set to 10000; the exploration rate is 0.99; the batch size is 512; the discount factor is 0.95, influencing the agent's long-term planning and decision-making. the two fully connected layers are set to 128 units, determining the capacity of the neural network, and the network update interval is set to 200 steps, which helps balance the frequency of model parameter updates with system stability. The environmental parameters set for the baseline algorithm, PC-ID3QN, include 30 vehicles, an average content size of 8MB, and 50 content items.

B. Performance Metrics

To demonstrate the effectiveness of the algorithm, we evaluate the caching system based on three performance metrics: cache hit ratio (CHR), content transmit delay and system transmission cost.

1) CHR: The ratio of content cached by the edge server to the total content requests, which is a key indicator of cache performance [26].

2) Content Transmission Delay: The time taken to transmit content to the cache node.

3) System Engry Cost: The energy cost incurred to transmit content to the request node.

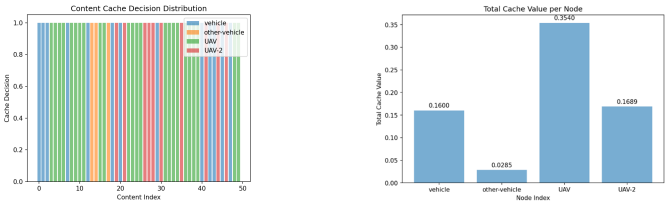
C. Reference Algorithm

We compare the proposed algorithm primarily with the following algorithms.

1) PC-DDQN: This improves upon DQN by reducing overestimation bias in Q-value updates by using two separate networks—one to select actions and another to evaluate them.

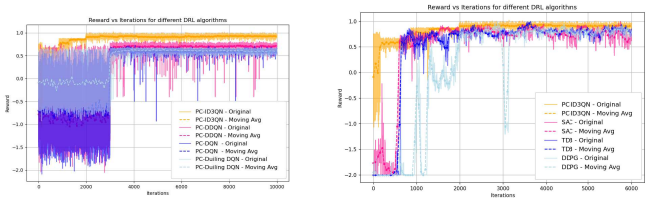
2) PC-Dueling DQN: This algorithm enhances DQN by separating the estimation of the state value and the advantage of each action.

3) RCA: Random Caching Algorithm.



(a) Cache placement (b) Cache value

Fig. 5. Theoretical solution–based caching strategy.



(a) discrete action (b) continuous action

Fig. 6. Average revenue in each episode with different DRL algorithms.

4) PCAZ: Zipf-Based Popular Caching Algorithm.

5) LRUCA: Least Recently Used Caching Algorithm.

6) SAC: This improves stability by adding entropy regularization to the policy, encouraging better exploration in continuous action spaces [27].

7) TD3: This enhances DDPG [29] by using double Q-networks, delayed policy updates, and target smoothing to reduce overestimation and improve learning stability [31].

8) Pre-Caching: cache the likely-to-be-requested data beforehand to reduce access delay. [24]

9) TCS: Threshold-based caching strategy: Set a cache threshold to evaluate the popularity of data. Only cache the data that exceeds this threshold [28].

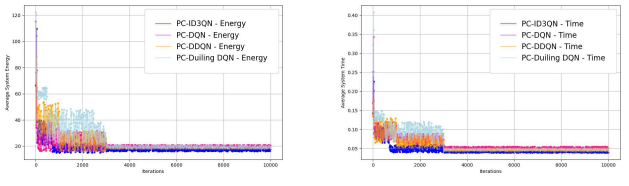
10) PCS: Probability-based caching strategy: Calculate a probability value based on the cache content's popularity, and use this probability to decide whether to cache the content [30].

D. Experimental Results and Analysis

In our experiments, we use system rewards as the metric to evaluate agent performance; higher rewards indicate better system performance. The experimental data is uniformly smoothed using moving average techniques, averaging every 50 points out of 10,000 iterations.

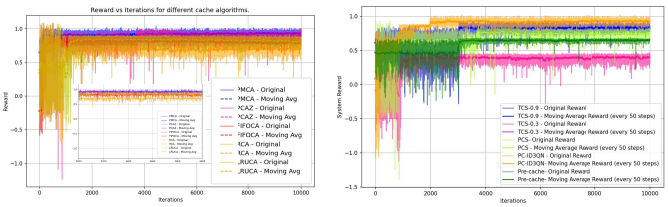
1) Performance Comparison: Fig 5(a) reflects the cache placement distribution of 50 requested contents across different nodes, while Fig 5(b) represents the value of cached content at each node. Through calculations $P_{1.3}$, we obtain the cache values for each cache node at the current time: 0.16 for vehicles, 0.0285 for adjacent vehicles, 0.354 for drones, and 0.1689 for adjacent drones. The total reward value is obtained by summing the values of each cache node. This result is close to the final conclusion obtained after the reinforcement learning model's iterative optimization, thus validating our theoretical derivation.

Fig. 6(a) shows the PC-ID3QN improves the system's average reward by 21.45 %, 31.43 %, and 33.48% over



(a) latency (b) energy cost

Fig. 7. Latency and energy cost in each episode with different DRL algorithm.



(a) Traditional algorithms (b) Advanced algorithms

Fig. 8. Average revenue in each episode with different cache algorithms.

the baseline methods (PC-DDQN, PC-Dueling DQN, and PC-DQN), respectively. DDQN still uses a single-stream Q network, it struggles to effectively separate state value from action impact, making its decision-making ability slightly inferior to D3QN. Dueling DQN, with its dueling structure, improves the adaptability of caching decisions but still relies on the traditional Q-value update mechanism. In contrast, DQN, due to its single Q network, suffers from severe Q-value overestimation and unstable learning. Fig. 6(b) further compares D3QN with continuous-action DRL algorithms such as DDPG, TD3, and SAC. Although these methods are designed to handle fine-grained action spaces and show moderate improvements in long-term utility, they require more complex policy and value updates, leading to slower convergence and reduced stability in highly dynamic edge environments. D3QN, operating in a discrete action space tailored to cache placement decisions and higher adaptability.

Fig. 7 shows the system's latency and energy consumption with different algorithms. All algorithms reduce latency and energy consumption to some extent, with PC-ID3QN consistently performing the best. This demonstrates the effectiveness of our training scheme. D3QN uses dual networks and targets to strategically cache based on content type and user characteristics, leading to faster convergence. Compared to the baseline methods (PC-DDQN, PC-Dueling DQN, and PC-DQN), PC-ID3QN reduces latency by 1.03%,1.73% and 2.21%, and energy consumption by 38.8%, 19.6%, and 17.2%, respectively. Although the improvement in latency is modest compared to other reinforcement learning algorithms, it can significantly enhance network QoS in complex scenarios with many users and alleviate network pressure.

In Fig. 8(a) we compare our proposed User Preference Matching Hierarchical Caching Replacement Algorithm (PMCA) based on the D3QN algorithm with other caching algorithms. It can be observed that PMCA achieves the highest average system reward, outperforming the other algorithms. In terms of system reward, it improves by 25.05%, 19.13%, 11.01%, and 8.13%, respectively. This improvement

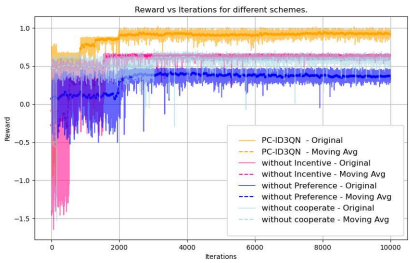


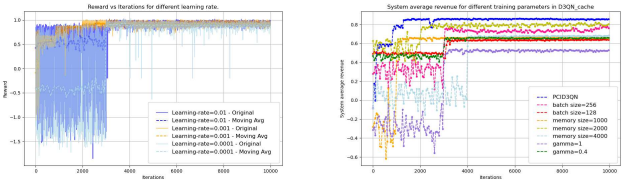
Fig. 9. Average revenue in each episode with different schemes.

is attributed to the consideration of user preferences, which enhances cache hit rates to a certain extent. In contrast, RCA, with its inherent randomness, shows weaker performance, leading to a reduced cache hit rate. PCAZ, utilizing the Zipf distribution, is more suitable for caching popular content, and thus, typically surpasses RCA in system rewards. LRUCA, despite retaining recently used content, underperforms the user preference-based algorithm in terms of system reward for less frequently used but important content.

From Fig. 8(b), shows the system utility under different caching algorithms based on D3QN. The green line denotes the Pre-caching strategy. Although the Pre-caching approach shows minimal fluctuation initially, its final system utility is relatively low due to its limitations because user content requests change over time. The cache advantages of the PCS and the TCS are far inferior to the algorithm we proposed. For the threshold-based caching method, when the cache threshold is too low, the system prematurely caches some less important or less frequently accessed files into the cache pool. This is leading to the cache pollution problem. When the cache threshold is set too high, this results in fresh content being cached slowly, and is easily deleted quickly because it has just arrived and its access frequency is not high. As for the probability-based caching strategy, it has a higher degree of randomness. It is causing unnecessary energy consumption.

Fig. 9 illustrates the average system reward and training iterations under different schemes based on the PC-ID3QN algorithm. The red line represents the proposed scheme in this paper, the pink line represents the proposed scheme without the incentive mechanism, the light green line depicts scenarios where there is no cooperation between vehicles and UAVs, which is akin to reducing cache capacity, and the blue line shows the proposed scheme with user preference-based caching decisions removed. The comparison results indicate that the non-user preference strategy yields the lowest average system reward among the four strategies, stabilizing around 0.42. Compared to the cooperative caching schemes, our proposed PC-ID3QN algorithm improves the average system reward by 28.4%. The absence of incentives suggests reduced cooperation among vehicles, resulting in a system reward similar to that of non-cooperative strategies. Additionally, removing user preference-based caching significantly decreases the system reward.

2) Sensitivity Analysis of Training Parameters: In our original algorithm, the batch size, learning rate, discount factor, and experience replay buffer were set to 512, 0.001, 0.99, and 3000, respectively.



(a) Learning rate (b) Other training parameters

Fig. 10. System average revenue for different training parameters in D3QN-cache.

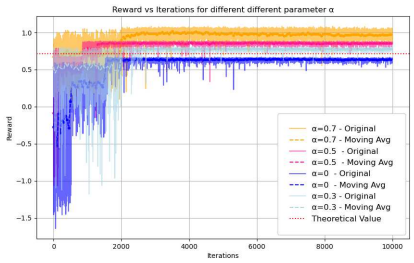
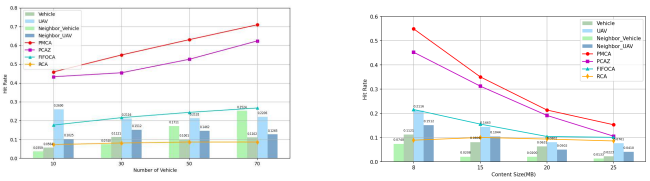


Fig. 11. Average revenue in each episode with different parameter α.

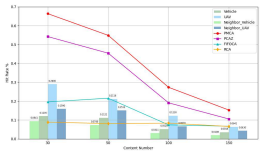
Fig. 10a shows the impact of different learning rates on the performance of the PC-ID3QN algorithm. We compared learning rates of 0.01, 0.001, and 0.0001. With a learning rate of 0.01, the algorithm may converge too quickly, leading to large fluctuations later and making it difficult to maintain stable performance. At a learning rate of 0.0001, training is stable but slow, with a risk of stagnation. In contrast, a learning rate of 0.001 allows the algorithm to achieve rapid convergence while maintaining high stability, offering the best balance. Therefore, 0.001 is the optimal choice for the PC-ID3QN algorithm, ensuring efficient and stable operation.

Considering computational resources, we experimented with batch sizes of 128 and 256. As shown in Fig. 10b, smaller batch sizes lead to a more unstable training process, causing larger gradient estimation noise because each update is based on fewer data points, thus increasing the overall training time. When the experience replay buffer is set too small (2000), the lack of sufficient historical experience may result in a lack of diversity in the training samples, preventing the agent from learning long-term strategies, which leads to low training efficiency. If too much experience (4000) is stored, it may include failure experiences caused by incorrect actions or undesirable environmental states. It may ultimately affecting policy optimization. A small discount factor (0.4) causes the agent to focus only on immediate rewards. This leads the agent to adopt a "short-sighted" strategy, focusing only on immediate gains while ignoring potential long-term benefits. If gamma is too large(gamma=1), the agent may overestimate future rewards, even making unrealistic or overly optimistic decisions, neglecting the current dynamic environment.

Fig. 11 shows the to verify the impact of the incentive mechanism on system utility, we conducted experiments with different values of α. Specifically, when $\alpha = 0$, the system's caching utility is approximately 0.6609, which is close to our theoretical optimal solution, i.e., our optimization goal. Then, we set different values of α (0.3, 0.5, 0.7) to verify whether the



(a) vehicle number (b) average content size



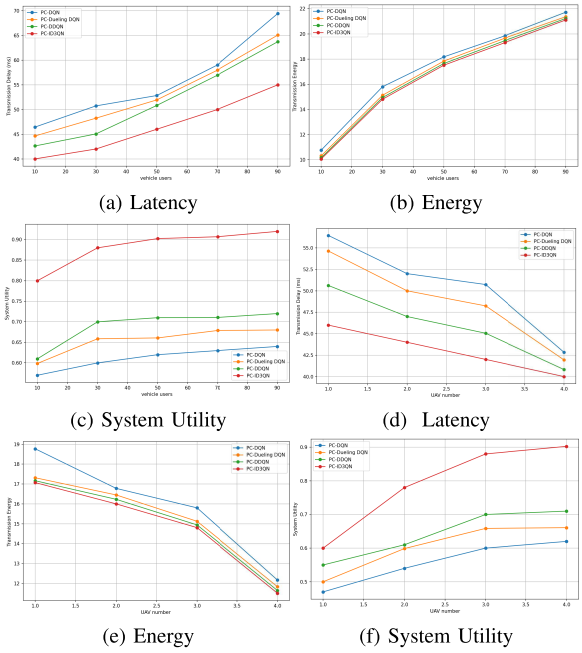
(c) content number

Fig. 12. Impact of environment variable on system performance.

proposed incentive mechanism could improve system utility. The experimental results show that when α increased from 0.3 to 0.5, the caching utility increased, demonstrating the positive effect of the incentive mechanism in improving caching utility. However, when α increased from 0.5 to 0.7, the increase in system caching utility became very small. This is because the vehicle group is limited, and the system utility is already close to its limit, so there was not much improvement in system rewards. Therefore, we use the experimental incentive with $\alpha = 0.5$, and further increasing α did not result in significant improvements.

3) Sensitivity Analysis of Environmental Parameters: Fig. 12(a) shows that as the number of vehicles increases from 10 to 70, the cache hit rate also increases. When the number of vehicles reaches 50, the hit rate of neighboring vehicles surpasses that of the vehicle itself due to higher vehicle density and increased contact probability. However, the hit rate of drones remains stable, as the increase in repeated requests offsets some of the benefits. Fig. 12(b) illustrates the impact of content size on cache hit probability. Due to limited cache capacity, the hit probability decreases as content size increases, reducing the number of effectively cached items. This effect is more pronounced for vehicles and neighboring vehicles than for drones, as vehicles have more constrained cache capacity. Fig. 12(c) shows that as the number of requested content items increases, user preferences become more diverse, and content popularity becomes more dispersed. However, with limited cache capacity, hit rates are higher when there are fewer content types. The line graph indicates that when the number of content items reaches 100, FIFOCA achieves a slightly higher cache hit rate than random caching. This is because the variability in content types results in lower hit rates for both RCA and FIFOCA.

4) Robustness and Scalability Analysis: Fig. 13(a) (b) (c), we illustrate the impact of the number of vehicle users on the algorithm in terms of average system delay, system energy consumption, and system utility. From the figures, we can see that both the system's average delay and energy consumption increase as the number of users grows. This is due to the limited cache resources—when the number of users increases,



(a) Latency (b) Energy

(c) System Utility (d) Latency

(e) Energy (f) System Utility

Fig. 13. Comparison under different number of vehicle users and UAVs under different algorithms.

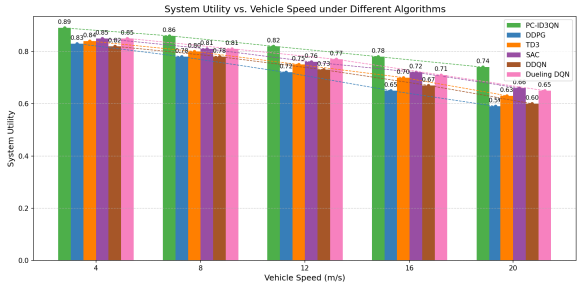


Fig. 14. Comparison of different vehicle speed.

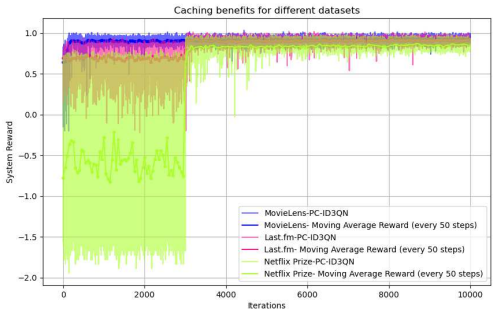


Fig. 15. Comparison of system utility across different datasets.

the content requested also increases. Although the additional users somewhat increase the available cache space at edge nodes, improving the cache hit rate, the overall system performance still worsens, and the system's cache utility declines. However, the proposed PC-ID3QN algorithm outperforms other baseline algorithms, showing better robustness and real-time adaptability.

Fig. 13(d) (e) (f) show the impact of UAV quantity on the algorithm. Based on the scenario we built, we only increase the number of UAVs in the figure, while keeping the number of users constant. As the number of UAVs increases, the available resources can better meet the needs of all users. From the figures, we can see that as the number of UAVs increases, the average system delay decreases, and energy consumption decreases. This is because adding more edge cache nodes reduces the chances of fetching content from the base station. The system utility starts to flatten as all users' needs are already met, and the remaining resources no longer provide a significant improvement for users. However, in practice, too many UAVs may cause interference in wireless channels, leading to overlapping frequencies and decreased efficiency. Therefore, the number of UAVs should not be set too high.

Shown in the Fig. 14, we compare the system utility of six algorithms at different vehicle speeds (4 m/s to 20 m/s). The overall system utility of all algorithms shows a downward trend when the vehicle speed increases. The PC-ID3QN algorithm achieves the highest system utility in all speed scenarios, from 0.89 at 4 m/s to 0.74 at 20 m/s, outperforming other algorithms with the slowest downward trend. This indicates that the method has stronger adaptability and robustness to user request patterns and environmental changes. DDPG and TD3 performed relatively poorly, especially at 16 m/s and 20 m/s, with significant declines in system utility (as low as 0.59 and 0.60), indicating the limitations of continuous control methods in handling discrete caching decisions and highly dynamic environments.

Fig. 15 shows the verification results of system utility using different datasets, with the same reinforcement learning method (D3QN) and the same caching optimization algorithm applied in the experiments. The parameters are the same as those used in the benchmark algorithm. By comparing the system's performance on the MovieLens and Last.fm datasets, we found that the trend of system utility changes is generally consistent across different datasets, demonstrating that the proposed method has good generality and scalability. Since the content and processing methods of the Netflix Prize and MovieLens datasets are quite similar, the experimental results are also the most similar.

VII. Conclusion

In this paper, we consider user preferences and global content popularity, combine the whole process of content caching and replacement, and design a collaborative caching algorithm based on D3QN. First, we design a network framework based on edge caching, defining dynamic content caching and replacement decisions between edge nodes based on individual preferences. Next, we formulate an optimization problem to maximize the overall system utility, which was solved and cross-validated through both theoretical and reinforcement learning approaches. and obtain the simulation solution using the proposed PC-ID3QN algorithm. Finally, simulation results validate the effectiveness of the algorithm in improving system performance. Compared to DDQN, Dueling DQN, and DQN, our proposed algorithm reduces request latency by 1.03%,1.73% and 2.21%, respectively, and energy consumption by 38.8%, 19.6%, and 17.2%. In future research, we can explore pre-caching algorithms based on traffic flow.

APPENDIX
PROOF OF THE MULTIPLE BACKPACK PROBLEM

In this section, we rigorously prove through mathematical analysis that the problem of maximizing user service quality is NP-hard.

Lemma 1: P_1 is NP-hard.

The central concept in proving that a problem is NP-hard involves finding a reduction process. This reduction establishes a connection between two algorithms. Our objective is to map an instance of any known NP-hard problem to an instance of our problem. Based on the transitivity of reductions, as referenced in literature [18], we can then conclude that our problem is NP-hard. The proof is as follows:

We choose to map the Multiple Knapsack Problem (MKP) to our problem. The MKP is a well-known NP-hard problem, where the objective is to select items within a set of knapsacks to maximize the total value of the items, while adhering to the capacity constraints of each knapsack. This problem has been extensively studied in the literature [19].

$$\begin{aligned}
&\max imize \sum_{m=1}^{M}\sum_{f=1}^{F} p_f x_{m,f}\\
&\text{subject to:}\quad C_1': \sum_{f=1}^{F} \omega_f x_{m,f} \le L_m, m \in M,\\
&\qquad\qquad C_2': \sum_{m=1}^{M} x_{m,f} \le 1, f \in F,\\
&\qquad\qquad C_3': x_{m,f} \in \{0,1\}, m \in M, f \in F
\end{aligned}$$

Let M represent the number of knapsacks and F the number of items. The parameters w_f and p_f denote the weight and profit of item f, respectively. The parameter L_m represents the capacity of the m-th knapsack under constraint C_1'. The decision variable x_{fm} is set to 1 if and only if item f is assigned to the m-th knapsack. The constraint $\sum_{m=1}^{M} x_{fm} \le 1$ ensures that each item f is assigned to at most one knapsack, where x_{fm} represents a binary decision variable.

Transform the instance of the MKP problem into an instance of the P1 problem: Treat each knapsack capacity in the MKP problem as the cache capacity of vehicles and UAVs in our problem. Treat each item in the MKP problem as a content item in problem P_1, and map the weight and profit of each item in the MKP problem to the cache size and cache value of each content item in problem P_1.

Given the parameters above, we construct an instance of the problem with M users and F items. For each user m (where vehicles and UAVs are collectively considered as users), and for each content $f \in F$, let $C_{\mathrm{M}} \triangleq L_m, \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} = 1, s_f \triangleq \omega_f, V \cup U \triangleq \mathrm{M}, G_f \triangleq F$.

Question P_1 can be written as:

$$\begin{aligned}
&{}_1^{+}: \max_{c_{mf}^t} U\\
&\quad\text{subject to:}\quad s.t. C_1^{+}: c_{mf}^t \in \{0,1\}, \forall f, m \in F, M\\
&\qquad\qquad C_3^{+}: \sum_{f=1}^{F} c_{mf}^t \omega_f \le L_m, \quad \forall m \in M,\ \forall f \in F\\
&\qquad\qquad C_4 - \mathrm{C}_5
\end{aligned}$$

Since the multiple knapsack problem does not involve the energy consumption related to the movement of knapsacks, we assume fixed coordinates for vehicles and UAVs and constant transmission power in our subsequent proof. Therefore, the transmission delay $D_{f,v}^{total}$ experienced by users also needs to be rewritten as $D_{f,v}^{total} = c_{mf}^t d_m^f$.

Question P_1^+ can be written as:

$$\begin{aligned}
&P_2^+ :\\
&\max_{c_{mf}^t} \sum_{f=1}^{F}\sum_{m=1}^{M} \frac{p_f(d_{b,v}^f - c_{vf}^t d_m^f)}{d_{b,v}^f} + \frac{p_f(\varpi_b E_{bv}^f - p_m \varpi_m c_{vf}^t d_m^f)}{\varpi_b E_{bv}^f}\\
&s.t. C_1^{+} C_3^{+} C_4
\end{aligned}$$

The objective function includes a linear combination of two terms. In a given environment, terms ϖ_b, p_m and ϖ_m can be considered constants. Therefore, the formulation of cache energy efficiency is equivalent to that of cache time efficiency. The solution to the cache time efficiency part is a crucial step in solving the entire problem, and the complexity of the entire problem depends on the computational complexity of this part. Thus, if we reduce the time efficiency problem to the multiple knapsack problem, we can demonstrate that the entire problem is NP-hard. In this case, the objective function for P_2^+ becomes:

$$\begin{aligned}
P_3^+ : &\max_{c_{mf}^t} \sum_{f=1}^{F}\sum_{m=1}^{M} \frac{p_f(d_{b,v}^f - c_{vf}^t d_m^f)}{d_{b,v}^f}\\
&s.t. \quad C_1^{+} C_3^{+} C_4
\end{aligned}$$

According to the problem statement, the objective function P_3^+ can be expressed in terms of $p_f + \frac{p_f c_{mf}^t d_m^f}{d_{b,v}^f}$. Since $0 < \frac{d_m^f}{d_{b,v}^f} \le 1$ is a constant and does not affect the complexity, we can extract and rewrite the relevant part of the objective function P_3^+ in the following form:

$$\begin{aligned}
P_4^+ : &\max_{c_{mf}^t} \sum_{f=1}^{F}\sum_{m=1}^{M} p_f c_{mf}^t\\
&s.t. \quad C_1^{+}: c_{mf}^t \in \{0,1\}, \forall f, m \in F, M\\
&\qquad C_3^{+}: \sum_{f=1}^{F} c_{mf}^t \omega_f \le L_m, \forall m \in M, \forall f \in F\\
&\qquad C_4: \sum_{f=1}^{F} c_{mf}^t \le 1, \forall f \in F
\end{aligned}$$

For all users m and content f, the c_{mf}^t in problem P_4^+ and $x_{m,f}$ in the MKP problem are equivalent. Therefore, a polynomial-time reduction is constructed to map an instance of the MKP problem to an instance of problem P_1.

In conclusion, due to this transformation and the fact that the MKP problem is NP-hard, we can infer by transitivity that problem P_1 is NP-hard.

REFERENCES

[1] L. Yao, Y. Wang, X. Wang, and G. WU, "Cooperative caching in vehicular content centric network based on social attributes and mobility," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 391–402, Feb. 2021.

[2] B. Chen and C. Yang, "Caching policy for cache-enabled D2D communications by learning user preference," *IEEE Trans. Commun.*, vol. 66, no. 12, pp. 6586–6601, Dec. 2018.

[3] G. Yu, Y. He, J. Wu, Z. Chen, and J. Pan, "Mobility-aware proactive edge caching for large files in the Internet of Vehicles," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11293–11305, Jul. 2023.

[4] Y. Liu, C. Yang, X. Chen, and F. Wu, "Joint hybrid caching and replacement scheme for UAV-assisted vehicular edge computing networks," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 1, pp. 866–878, Jan. 2024.

[5] M. Alzenad, A. El-Keyi, F. Lagum, and H. Yanikomeroglu, "3-D placement of an unmanned aerial vehicle base station (UAV-BS) for energy-efficient maximal coverage," *IEEE Wireless Commun. Lett.*, vol. 6, no. 4, pp. 434–437, Aug. 2017.

[6] C. Jiang, L. Gao, J. Luo, P. Zhou, and J. Li, "A game-theoretic analysis of joint mobile edge caching and peer content sharing," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1445–1461, May 2023.

[7] D. Zhang, W. Wang, J. Zhang, T. Zhang, J. Du, and C. Yang, "Novel edge caching approach based on multi-agent deep reinforcement learning for Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 8, pp. 8324–8338, Aug. 2023.

[8] C. Hou, C. Zhou, Q. Huang, and C.-B. Yan, "Cache control of edge computing system for tradeoff between delays and cache storage costs," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 1, pp. 827–843, Jan. 2024.

[9] Y. Guan, X. Zhang, and Z. Guo, "PrefCache: Edge cache admission with user preference learning for video content distribution," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 4, pp. 1618–1631, Apr. 2021.

[10] Z. Zhang, M. St-Hilaire, X. Wei, H. Dong, and A. E. Saddik, "How to cache important contents for multi-modal service in dynamic networks: A DRL-based caching scheme," *IEEE Trans. Multimedia*, vol. 26, pp. 7372–7385, 2024.

[11] W. Chaowei, W. Ziye, X. Lexi, Y. Xiaofei, Z. Zhi, and W. Weidong, "Collaborative caching in vehicular edge network assisted by cell-free massive MIMO," *Chin. J. Electron.*, vol. 32, no. 6, pp. 1218–1229, Nov. 2023.

[12] C. Li, Y. Zhang, and Y. Luo, "A federated learning-based edge caching approach for mobile edge computing-enabled intelligent connected vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 3, pp. 3360–3369, Mar. 2023.

[13] X. Huang, K. Xu, Q. Chen, and J. Zhang, "Delay-aware caching in Internet-of-Vehicles networks," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10911–10921, Jul. 2021.

[14] R. D. Yates, "The age of gossip in networks," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2021, pp. 2984–2989.

[15] G. Chen, S. Qi, F. Shen, Q. Zeng, and Y.-D. Zhang, "Information-aware driven dynamic LEO-RAN slicing algorithm joint with communication, computing, and caching," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 5, pp. 1044–1062, May 2024.

[16] Z. Yu, J. Hu, G. Min, Z. Zhao, W. Miao, and M. S. Hossain, "Mobility-aware proactive edge caching for connected vehicles using federated learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5341–5351, Aug. 2021.

[17] W. Murray and K.-M. Ng, "An algorithm for nonlinear optimization problems with binary variables," *Comput. Optim. Appl.*, vol. 47, no. 2, pp. 257–288, Oct. 2010.

[18] D.-H. Tran, S. Chatzinotas, and B. Ottersten, "Satellite- and cache-assisted UAV: A joint cache placement, resource allocation, and trajectory optimization for 6G aerial networks," *IEEE Open J. Veh. Technol.*, vol. 3, pp. 40–54, 2022.

[19] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Germany: Springer-Verlag, 2004.

[20] J. Chen, H. Wu, P. Yang, F. Lyu, and X. Shen, "Cooperative edge caching with location-based and popular contents for vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 9, pp. 10291–10305, Sep. 2020.

[21] J. Shi, L. Zhao, X. Wang, W. Zhao, A. Hawbani, and M. Huang, "A novel deep Q-learning-based air-assisted vehicular caching scheme for safe autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4348–4358, Jul. 2021.

[22] Y. Yang, Z. Liu, Z. Liu, Y. Xie, K. Y. Chan, and X. Guan, "Joint optimization of edge computing resource pricing and wireless caching for blockchain-driven networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 6, pp. 6661–6670, Jun. 2022.

[23] H. Li, M. Sun, F. Xia, X. Xu, and M. Bilal, "A survey of edge caching: Key issues and challenges," *Tsinghua Sci. Technol.*, vol. 29, no. 3, pp. 818–842, Jun. 2024.

[24] Y. Liu, J. Jia, J. Cai, and T. Huang, "Deep reinforcement learning for reactive content caching with predicted content popularity in three-tier wireless networks," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 1, pp. 486–501, Mar. 2023.

[25] B. Liu, C. Liu, and M. Peng, "Dynamic cache placement and trajectory design for UAV-assisted networks: A two-timescale deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 73, no. 4, pp. 5516–5530, Apr. 2024.

[26] H. Wu, B. Wang, H. Ma, X. Zhang, and L. Xing, "Multiagent federated deep-reinforcement-learning-based collaborative caching strategy for vehicular edge networks," *IEEE Internet Things J.*, vol. 11, no. 14, pp. 25198–25212, Jul. 2024.

[27] Z. Zhang, C.-H. Lung, X. Wei, M. Chen, S. Chatterjee, and Z. Zhang, "In-network caching for ICN-based IoT (ICN-IoT): A comprehensive survey," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14595–14620, Aug. 2023.

[28] L. Leira, M. Luís, and S. Sargento, "Context-based caching in mobile information-centric networks," *Comput. Commun.*, vol. 193, pp. 214–223, Sep. 2022.

[29] H. Zhou, H. Wang, Z. Yu, G. Bin, M. Xiao, and J. Wu, "Federated distributed deep reinforcement learning for recommendation-enabled edge caching," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 3640–3656, Nov. 2024.

[30] D. Gupta, S. Rani, A. Singh, and J. J. Rodrigues, "ICN based efficient content caching scheme for vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15548–15556, Dec. 2023.

[31] Z. Hu, C. Fang, R. Zhong, and Y. Liu, "Joint physical and network layers design for STARS-assisted multi-cellular edge caching," *IEEE Trans. Wireless Commun.*, vol. 23, no. 11, pp. 17446–17460, Nov. 2024.

[32] R. Farahani, E. Çetinkaya, C. Timmerer, M. Shojafar, M. Ghanbari, and H. Hellwagner, "ALIVE: A latency- and cost-aware hybrid P2P-CDN framework for live video streaming," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 2, pp. 1561–1580, Apr. 2024.

[33] R. Farahani, M. Shojafar, C. Timmerer, F. Tashtarian, M. Ghanbari, and H. Hellwagner, "ARARAT: A collaborative edge-assisted framework for HTTP adaptive video streaming," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 1, pp. 625–643, Mar. 2023.

[34] M. Rahmani, F. Delavernhe, S. M. Senouci, and M. Berbineau, "Toward sustainable last-mile deliveries: A comparative study of energy consumption and delivery time for drone-only and drone-aided public transport approaches in urban areas," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 17520–17532, Nov. 2024.

[35] G. Chen, X. Mu, H. Liang, Q. Zeng, and Y.-D. Zhang, "Distributed RAN slicing based on MATD3 joint with evolutionary game assisted user association for MEC-enabled HetNets," *IEEE Trans. Wireless Commun.*, vol. 24, no. 1, pp. 260–276, Jan. 2025.

[36] G. T. Maale, G. Sun, N. A. E. Kuadey, T. Kwantwi, R. Ou, and G. Liu, "DeepFESL: Deep federated echo state learning-based proactive content caching in UAV-assisted networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12208–12220, Sep. 2023.

Geng Chen (Member, IEEE) received the B.S. degree in electronic information engineering and the M.S. degree in communication and information system from Shandong University of Science and Technology, Qingdao, China, in 2007 and 2010, respectively, and the Ph.D. degree in information and communications engineering from Southeast University, Nanjing, China, in 2015. He is currently a Full Professor with the College of Electronic and Information Engineering, Shandong University of Science and Technology. His current research interests include heterogeneous networks, ubiquitous networks, and software defined mobile networks, with emphasis on wireless resource management and optimization algorithms, and precoding algorithms in large scale MIMO.

Yuchen Han received the B.S. degree in physics from Qilu Normal University, Jinan, China, in 2023. She is currently pursuing the master's degree in information and communication engineering with Shandong University of Science and Technology. Her current research interests include vehicular heterogeneous networks and edge caching, with a focus on edge caching strategy optimization and collaborative caching.

Fei Shen (Member, IEEE) received the bachelor's degree in information technology from Southeast University, Nanjing, China, the master's degree in communications technology from Ulm University, Ulm, Germany, and the Ph.D. degree from Dresden University of Technology (TU Dresden), Dresden, Germany. She was a Research Associate with Chinese University of Hong Kong (CUHK) from 2007 to 2008 and Ulm University from 2008 to 2009. Since 2010, she has been a Scientific Research Associate and then as a Post-Doctoral Researcher with TU Dresden, accomplishing the Focus Project "COIN" of German Research Foundation (DFG). From 2015 to 2017, she worked for the Starting Grant "MORE" of European Research Council (ERC), as a Post-Doctoral Researcher and a Senior Research Engineer with CentraleSupelec, and Mine-Telecom, ParisTech, France. She is currently a Professor with Shanghai Institute of Microsystem and Information Technology, Chinese Academy of Sciences. Her current research interests include resource optimization for wireless communications, edge and fog computing.

Qingtian Zeng (Member, IEEE) received the B.S. and M.S. degrees in computer science from Shandong University of Science and Technology, Taian, China, in 1998 and 2001, respectively, and the Ph.D. degree in computer software and theory from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2005. He is currently a Professor with Shandong University of Science and Technology, Qingdao, China. His research interests are in the areas of Petri nets, process mining, and knowledge management.

Yu-Dong Zhang (Senior Member, IEEE) was a Post-Doctoral Researcher with Columbia University, USA, from 2010 to 2012, and an Assistant Research Scientist with the Research Foundation of Mental Hygiene (RFMH), USA, from 2012 to 2013. He was a Full Professor with Nanjing Normal University from 2013 to 2017. He is currently a Professor with the School of Computing and Mathematical Sciences, University of Leicester, U.K. His research interests include deep learning and medical image analysis. He is a fellow of IET, EAI, and BCS; a Senior Member of IES and ACM; and a Distinguished Speaker of ACM. He was included in the Most Cited Chinese Researchers (Computer Science) by Elsevier from 2014 to 2018. He was the 2019 and 2021 recipient of a Highly Cited Researcher by Clarivate. He won the Emerald Citation of Excellence 2017 and MDPI Top10 Most Cited Papers 2015. He is included in the Top Scientist in Research.com.